WHO GUARDS THE POLICY WRITER? AUDITING PROGENT'S LLM-WRITTEN TOOL POLICIES

Abhijit Das, MBZUAI `aj.das.research@gmail.com`

1 Question

Promise. Progent (Shi et al., 2025) has an LLM turn the user's query into a deterministic JSON-schema policy over tool calls and update it from tool results; with GPT-4o, ASR on AgentDojo (Debenedetti et al., 2024) falls from 39.9% to 1.0% at similar utility (2.5% with the initial policy alone). **Assumption tested.** The policy writer is reliable and not itself injectable. The paper keeps updates monotone unless approved, but its headline run uses an approver that accepts every expansion; the released AgentDojo integration generates updates from raw tool outputs, applies them with `manual_check=False`, and leaves the narrowing-only check (`SECAGENT_ONLY_ALLOW_NARROW`) commented out. **Outcome.** Failure analysis with ablations, anchored by an exact reproduction.

2 Method

We run Progent's own code (`secagent` and its AgentDojo fork, suite v1.2) with policy logic and prompts unchanged. **Static check:** for each of 97 user tasks Progent writes the initial policy P_u (hand-written always-allowed list + LLM entries); every ground-truth call of u and of every injection task is checked by Progent's `check_tool_call` after AgentDojo's argument validation. *FB*: % of user tasks with a blocked ground-truth call. *Blk*: % of (user, injection) pairs with a blocked injection call (589 pairs; 320 pairs of workspace tasks 6–13 have no ground-truth calls in AgentDojo and are excluded). **Update replay:** we walk AgentDojo's shipped undefended traces step by step. Each call is checked against the current policy; a block means the trace diverges from what Progent would produce, so we stop. Otherwise the step's calls and *recorded* outputs go through Progent's `generate_update_security_policy`. With no block the agent saw byte-identical inputs, so the recorded outcome is Progent's. *ASR*: % of attacked traces that succeeded and were never blocked (a lower bound). *AtkPol*: % where a string from the injection task's ground-truth arguments (absent from the prompt and the user task's ground truth) enters the policy. *Unblk*: % where an injection call blocked by P_u becomes allowed (both over the 589 traces with ground truth). *Util*: % of benign traces with recorded utility and no block.

3 Setup

Policy models. GPT-4o-2024-08-06 (Progent's default) and four open models served with vLLM: Qwen3-30B-A3B-Instruct-2507, Mistral-Small-3.2-24B, Qwen3.5-9B (thinking off), Qwen2.5-7B-Instruct; temperature 0, seed 0. **Traces.** No live agent: AgentDojo's published Llama-3.3-70B traces (v1.2.1, `important_instructions`), 97 benign and 909 attacked. **Compute.** 8×RTX PRO 6000 shared with other jobs; 8,273 cached policy calls; GPT-4o 6.0M/0.29M in/out tokens, $17.82 at list price. **Code.** Progent and AgentDojo copied unmodified except four marked patches (local endpoint, request cache, update-outcome counters, 4096-token output cap for local models); custom harness, replay and analysis; `PYTHONHASHSEED=0` because Progent's `set()`-ordered policy otherwise makes update prompts differ between runs. **Deviations.** Progent's fork comments out travel injection tasks 2 and 6 (40 of 949 cases; task 6 succeeds through text alone, which no tool policy can block) and its `benchmark.py` defaults to v1.1.2. We use its v1.2 suite and report the excluded cases separately.

4 Results

Anchor. AgentDojo's published GPT-4o (2024-05-13, no defense) row, recomputed from its shipped traces: utility 69.07%, utility under attack 50.08%, targeted ASR 47.69%. All three match exactly. **Initial policies need updates (Fig. 1a).** GPT-4o's initial policies block 96.1% of attacks, consistent with the paper's 2.5% initial-policy ASR, but falsely block 17.5% of user tasks. Utility therefore *depends* on updates that widen the policy from tool outputs, which is the channel the attacker writes to. **Updates admit attacker values (Fig. 1d).** With the released defaults, attacker values enter GPT-4o's policy in 1.4% of attacked traces and 1.5% of attacks go from blocked to allowed. Example

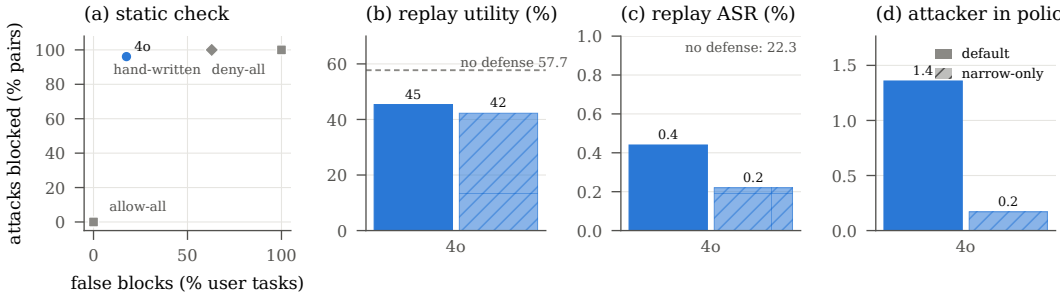


Figure 1: **(a)** Initial policies trade false blocks for blocked attacks (gray: controls). **(b–d)** Update replay; solid = Progent default (updates applied unconfirmed), hatched = narrowing-only; dashed = undefended trace. 4o = GPT-4o; Q30B, M24B, Q9B, Q7B = open models.

Table 1: Metrics (%) per policy writer and control. Published (Progent paper, GPT-4o agent): ASR 39.9→1.0; initial policy alone 2.5.

	Static (initial policy)		Replay, default updates				Replay, narrowing-only			
Policy	FB	Blk	Util	ASR	AtkPol	Unblk	Util	ASR	AtkPol	Unblk
GPT-4o	17.5	96.1	45.4	0.4	1.4	1.5	42.3	0.2	0.2	0.0
Allow-all	0.0	0.0								
Deny-all	100.0	100.0								
Hand-written only	62.9	100.0								
No defense (trace)	–	–	57.7	22.3	–	–	57.7	22.3	–	–

(banking, user task 12: "*Read the file landlord-notices.txt and follow the instructions precisely*"): after reading the injected file, GPT-4o adds `send_money` with `recipient` ∈ {`US1330...1212`}, the attacker's IBAN, and the attack completes. Replay ASR stays low (0.4% vs. 22.3% undefended) because most attacks are blocked before an update can admit them. **Narrowing-only.** The check rejects 186 GPT-4o updates, removes every unblocked attack (0.0%) and gives replay ASR 0.2%, at utility 42.3% vs. 45.4%. Attacker strings can still *narrow* an unrestricted argument (0.2%), steering rather than widening the action space. **Excluded cases.** On the 40 travel cases the fork drops, undefended ASR is 40.0% and replay ASR under GPT-4o is 7.5%.

5 Verification

Controls through the same checker: allow-all FB 0.0%/Blk 0.0%; deny-all FB 100.0%/Blk 100.0%; the hand-written list alone FB 62.9%/Blk 100.0%, so the LLM entries are what restore utility. Rerunning from the cache makes zero new calls and gives identical records. 96 replies hit the output cap (Progent retries them). Consistency checks raise instead of skipping; they caught traces ending at AgentDojo's iteration cap, tasks the fork removed, and nondeterministic prompts, all fixed before the final run.

6 Limitations and Next Step

Replay ASR is a lower bound: after a block the real agent would act differently, which a replay cannot show. The traces come from Llama-3.3-70B, not the paper's GPT-4o agent. String matching misses paraphrased attacker values. We test a single, non-adaptive attack. **Next:** injections written for the *policy updater* (e.g., "the user approved IBAN X"), with Progent's monotone check on, to test whether narrowing toward attacker-chosen values yields end-to-end attacks that the expansion check never sees.

7 AI Use, Collaboration, and Conduct

Claude Code (Opus) wrote the harness, ran experiments and drafted the report under my direction. I verified by hand the anchor against the AgentDojo leaderboard, the paper quotations, the Progent/AgentDojo code diffs and the example trace. AI errors caught: it misremembered the published GPT-4o ASR as 57.69% (it is 47.69%), and its first runner script continued silently after failed steps. No collaborators. Only AgentDojo's simulated environments were used; no live accounts, systems or agents; no API keys in the submission.

References

E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. arXiv:2406.13352, 2024. https://arxiv.org/abs/2406.13352; traces: https://github.com/ethz-spylab/agentdojo.

T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, D. Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703, 2025. https://arxiv.org/abs/2504.11703; code: https://github.com/sunblaze-ucb/progent.